SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: ITA 0607 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptron's,*

Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)

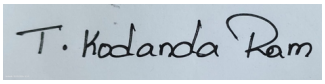
Assessment Tool Weightage: 10%

QUESTIONS:15 **Total Marks:25**

Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:
I T . KODANDA RAM (192524162) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Annexure A

Short Answer Test

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence. (5 Marks)

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance. (5 Marks)

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed. (5 Marks)

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capabilit y.
(5 Marks)

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization. (5 Marks)

6. A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy. (5 Marks)

7. A reinforcement learning agent fails to learn an optimal policy in a simulated environment. Investigate causes such as poor reward design, insufficient exploration, or incorrect state representation. Explain debugging methods and recommend tuning improvements like reward shaping, exploration strategies, and hyperparameters. (5 Marks)

8. A clustering algorithm like K-means produces inconsistent cluster assignments for similar datasets. Analyze possible reasons such as poor initialization, incorrect number of clusters, or sensitivity to outliers. Propose debugging approaches and optimization strategies like K-means++, normalization, and silhouette analysis. (5 Marks)

9. A support vector machine (SVM) model underperforms on a nonlinear dataset. Identify issues such as incorrect kernel choice, improper parameter tuning, or scaling problems. Suggest debugging techniques and optimization methods including kernel selection, hyperparameter tuning, and feature scaling. (5 Marks)

10. A natural language processing (NLP) model using recurrent neural networks (RNNs) suffers from poor long-term dependency learning. Examine causes such as vanishing gradients and limited memory capacity. Describe debugging steps and propose solutions like LSTM/GRU units, attention mechanisms, and improved training strategies. (5 Marks)

11. A support vector machine (SVM) model trained for classification is producing very low accuracy on both training and test datasets. Analyze possible causes such as improper kernel selection, incorrect hyperparameter tuning (C and gamma), or poor feature scaling. Describe systematic debugging steps to identify the issue and suggest optimization techniques like kernel tuning, normalization, and grid search to improve model performance. (5 Marks)

12. A K-Means clustering algorithm is applied to a dataset, but the clusters formed are highly imbalanced and do not reflect the actual data distribution. Investigate possible issues such as incorrect choice of K, poor initialization of centroids, or presence of outliers. Explain debugging approaches and suggest improvements like the Elbow method, K-Means++, and outlier removal techniques. (5 Marks)

13. A recurrent neural network (RNN) designed for time-series prediction shows inconsistent outputs and fails to capture long-term dependencies. Analyze possible causes such as vanishing gradients, insufficient sequence length, or improper architecture design. Describe debugging steps and propose optimization techniques such as using LSTM/GRU units, gradient clipping, and sequence normalization. (5 Marks)

14. A Naïve Bayes classifier used for text classification performs poorly when applied to real-world data containing noisy and irrelevant features. Examine possible issues such as violation of feature independence assumption, improper preprocessing, or imbalanced dataset. Explain debugging strategies and suggest improvements like feature selection, smoothing techniques, and data balancing methods. (5 Marks)

15. A reinforcement learning agent trained to solve a navigation problem fails to learn an optimal policy and shows erratic behavior. Analyze possible causes such as sparse rewards, improper exploration-exploitation balance, or incorrect reward function design. Describe debugging approaches and recommend optimization strategies like reward shaping, epsilon decay tuning, and policy optimization methods to improve learning performance. (5 Marks)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**20**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**30**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**10**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. Perceptron Model — Failure to Converge

QUESTION

A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence. **(5 Marks)**

ANSWER

Possible causes

- The learning rate may be unsuitable, causing updates that are too large or too small.
- The bias may be omitted or updated incorrectly, and the weight-update rule may contain a sign or label error.
- Input features or labels may be encoded incorrectly. Poor initialization can delay learning, although a correctly implemented perceptron on linearly separable data should converge.

Debugging steps

- Verify that labels use the expected values (for example, –1 and +1) and check the prediction and update equations.
- Print the weights, bias, misclassified samples, and number of errors after each epoch.
- Test the implementation on a small, known linearly separable dataset and confirm that the bias is updated whenever required.

Optimization

- Choose a reasonable positive learning rate and standardize features when their scales differ greatly.
- Shuffle the training samples between epochs and stop when an epoch has no classification errors, while retaining a sensible maximum-epoch limit.
- Correct the update rule and bias handling before changing hyperparameters.

Conclusion

Checking the mathematics and implementation first is the most reliable way to restore convergence; suitable scaling and learning-rate tuning can then improve stability.

2. Multilayer Perceptron — Slow Convergence and Local Minima

QUESTION

While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance. **(5 Marks)**

ANSWER

Possible causes

- Sigmoid or tanh activations in deep networks can produce very small gradients; unsuitable activations can also restrict learning.
- Poor initialization may make activations or gradients too small or too large.
- A learning rate that is too low slows training, while one that is too high can cause oscillation or divergence.

Debugging steps

- Plot training and validation loss by epoch and check whether loss decreases, plateaus, or oscillates.
- Inspect gradient magnitudes layer by layer, activation distributions, and parameter values for vanishing or exploding gradients.
- Verify backpropagation, labels, loss function, input scaling, and train/validation preprocessing.

Optimization

- Use ReLU or a suitable alternative for hidden layers and Xavier/He initialization as appropriate.
- Normalize or standardize input features and consider batch normalization where suitable.
- Try Adam or another appropriate optimizer, tune the learning rate, and use a learning-rate scheduler.

Conclusion

Gradient and loss diagnostics help identify the bottleneck; correct initialization, scaling, activation functions, and optimizer settings can improve learning.

---

3. Genetic Algorithm — Inconsistent Hypothesis Search

QUESTION

A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed. **(5 Marks)**

ANSWER

Possible causes

- Chromosome encoding may not represent valid candidate hypotheses, or decoding may be incorrect.
- The fitness function may reward the wrong behavior or contain a calculation error.
- Strong selection pressure and low mutation can reduce diversity and cause premature convergence; excessive mutation can make search nearly random.

Debugging steps

- Check that each chromosome decodes to a valid hypothesis and manually verify fitness for sample candidates.
- Track best, mean, and worst fitness across generations and record population diversity.
- Repeat experiments with different random seeds and compare the distribution of final fitness values.

Optimization

- Tune mutation and crossover rates; use mutation to restore diversity when the population becomes too similar.
- Adjust selection pressure to avoid eliminating useful alternatives too early.
- Use elitism to preserve the best candidates, while keeping enough population diversity and a suitable stopping condition.

Conclusion

Correct encoding and fitness evaluation are essential; diversity-aware parameter tuning can make results more reliable and improve search quality.

---

4. Deep Neural Network with Genetic Programming — Overfitting and High Cost

QUESTION

A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability. **(5 Marks)**

ANSWER

Possible causes

- A model with too many parameters or an overly complex evolved program can memorize training examples.
- The training dataset may be too small or unrepresentative of real cases.
- Evaluating many large candidate models in genetic programming can consume substantial time and computing resources.

Debugging steps

- Compare training and validation loss/accuracy. A large performance gap suggests overfitting.
- Inspect the number of parameters, evolved program size, model depth, and runtime per candidate.
- Check data quality, leakage, and whether the validation set represents unseen examples.

Optimization

- Apply L1/L2 regularization, dropout where appropriate, and early stopping based on validation performance.
- Prune unnecessary neural-network parameters or simplify oversized genetic-programming expressions.
- Use smaller candidate models, reuse cached evaluations where valid, and evaluate candidates on a representative subset before full evaluation.

Conclusion

Validation-based diagnosis, model simplification, regularization, and efficient candidate evaluation can reduce overfitting and computational cost.

5. Decision Tree — High Training Accuracy but Poor Test Performance

QUESTION

A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization. **(5 Marks)**

ANSWER

Possible causes

- A very deep tree may learn noise and details specific to the training set.
- Noisy labels, irrelevant features, or data leakage can create misleading training performance.
- The training and test data may come from different distributions.

Debugging steps

- Compare training, validation, and test metrics; inspect the confusion matrix and class-wise performance.
- Check for data leakage, duplicate records across splits, noisy labels, and distribution differences.
- Use cross-validation to estimate how consistently the tree performs on unseen data.

Optimization

- Prune the tree or restrict maximum depth, minimum samples per leaf, and minimum samples for a split.
- Select relevant features and handle noisy or missing data appropriately.
- Choose settings using cross-validation and evaluate the final model once on an untouched test set.

Conclusion

Reducing tree complexity and validating the model properly usually improves generalization when training performance is much higher than test performance.

---

6. Convolutional Neural Network — Unstable Training

QUESTION

A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy. **(5 Marks)**

ANSWER

Possible causes

- Excessive or label-changing data augmentation can make training examples unrealistic.
- Vanishing or exploding gradients can prevent stable parameter updates.
- Batch-normalization settings, small batch sizes, inconsistent preprocessing, or an excessive learning rate can cause fluctuations.

Debugging steps

- Inspect example images after augmentation and verify that labels remain correct.
- Plot training/validation loss and accuracy; check gradient norms and parameter values.
- Verify normalization layers, training/evaluation mode, input scaling, and consistent preprocessing.

Optimization

- Use realistic, label-preserving augmentation and consistent normalization.
- Apply suitable initialization and activations; use gradient clipping if gradients explode.
- Reduce or schedule the learning rate, and tune batch size or normalization strategy.

Conclusion

Checking the data pipeline, gradient flow, and normalization settings helps identify instability and supports more reliable CNN training.

---

7. Reinforcement Learning — Failure to Learn an Optimal Policy

QUESTION

A reinforcement learning agent fails to learn an optimal policy in a simulated environment. Investigate causes such as poor reward design, insufficient exploration, or incorrect state representation. Explain debugging methods and recommend improvements like reward shaping, exploration strategies, and tuning hyperparameters. **(5 Marks)**

ANSWER

Possible causes

- A reward may be sparse, misleading, or fail to reflect the intended goal.
- Insufficient exploration can prevent the agent from discovering useful actions; excessive exploration can slow exploitation.
- The state representation may omit information needed to choose actions, or learning parameters may be unsuitable.

Debugging steps

- Log episode rewards, episode lengths, action frequencies, and success rates over time.
- Inspect example states and transitions to verify that observations, actions, and rewards are correct.
- Check whether the agent visits relevant states and compare results across multiple random seeds.

Optimization

- Use carefully designed reward shaping that encourages progress without changing the intended objective.
- Choose an exploration strategy such as epsilon-greedy and tune its decay schedule.
- Tune learning rate, discount factor, and other algorithm-specific parameters; verify the state representation.

Conclusion

Correct rewards and states, sufficient exploration, and systematic hyperparameter tuning are key to improving policy learning.

8. K-Means — Inconsistent Cluster Assignments

QUESTION

A clustering algorithm like K-means produces inconsistent cluster assignments for similar datasets. Analyze possible reasons such as poor initialization, incorrect number of clusters, or sensitivity to outliers. Propose debugging approaches and optimization strategies like K-means++, normalization, and silhouette analysis. **(5 Marks)**

ANSWER

Possible causes

- K-Means can converge to different local solutions because it starts from different initial centroids.
- The selected number of clusters, K, may not suit the data.
- Features on large numeric scales can dominate distance calculations, and outliers can pull centroids.

Debugging steps

- Run the algorithm several times with different seeds and compare inertia and cluster assignments.
- Check feature scales, missing values, outliers, and whether the same preprocessing is used each time.
- Inspect cluster sizes and visualize the clusters where possible.

Optimization

- Use K-Means++ initialization and multiple initializations, retaining the best objective value.
- Standardize features when scales are not comparable.
- Use silhouette analysis, alongside domain knowledge, to assess candidate values of K.

Conclusion

Consistent preprocessing, better initialization, and an evidence-based choice of K reduce variability and improve cluster quality.

---

9. Support Vector Machine — Poor Performance on Nonlinear Data

QUESTION

A support vector machine (SVM) model underperforms on a nonlinear dataset. Identify issues such as incorrect kernel choice, improper parameter tuning, or scaling problems. Suggest debugging techniques and optimization methods including kernel selection, hyperparameter tuning, and feature scaling. **(5 Marks)**

ANSWER

Possible causes

- A linear kernel may not represent a nonlinear decision boundary.
- Unsuitable values of C or kernel-specific parameters can cause underfitting or overfitting.
- Unscaled features can dominate distance- or margin-based calculations.

Debugging steps

- Check label encoding, missing values, class balance, and preprocessing.
- Compare training and validation performance with a baseline model.
- Inspect whether feature scaling is fitted on training data only and applied consistently to validation/test data.

Optimization

- Standardize numerical features using a pipeline to avoid data leakage.
- Compare linear, polynomial, and RBF kernels where appropriate.
- Tune C and relevant kernel parameters using cross-validation and a structured search.

Conclusion

Appropriate scaling, kernel choice, and validation-based parameter tuning can improve an SVM's performance on nonlinear data.

---

10. RNN for NLP — Poor Long-Term Dependency Learning

QUESTION

A natural language processing (NLP) model using recurrent neural networks (RNNs) suffers from poor long-term dependency learning. Examine causes such as vanishing gradients and limited memory capacity. Describe debugging steps and propose solutions like LSTM/GRU units, attention mechanisms, and improved training strategies. **(5 Marks)**

ANSWER

Possible causes

- Gradients can shrink as they pass through many recurrent time steps, making early tokens difficult to learn from.
- A basic RNN may struggle to preserve useful information across long sequences.
- Truncated sequences, unsuitable preprocessing, or inadequate training can remove important context.

Debugging steps

- Check tokenization, padding, sequence lengths, and whether important context is being truncated.
- Monitor gradient norms and compare performance for short and long sequences.
- Inspect training/validation loss and test on examples where meaning depends on earlier context.

Optimization

- Use LSTM or GRU units, which are designed to retain information over longer sequences.
- Add an attention mechanism when appropriate so the model can focus on relevant tokens.
- Use suitable sequence lengths, gradient clipping when needed, and a tuned learning rate.

Conclusion

Sequence diagnostics and gated recurrent units or attention can help the model use context from earlier parts of the text.

---

11. Support Vector Machine — Low Training and Test Accuracy

QUESTION

A support vector machine (SVM) model trained for classification is producing very low accuracy on both training and test datasets. Analyze possible causes such as improper kernel selection, incorrect hyperparameter tuning (C and gamma), or poor feature scaling. Describe systematic debugging steps to identify the issue and suggest optimization techniques like kernel tuning, normalization, and grid search to improve model performance. **(5 Marks)**

ANSWER

Possible causes

- Low accuracy on both sets can indicate underfitting, incorrect labels, faulty preprocessing, or features with little predictive information.
- An unsuitable kernel or inappropriate C and gamma values can produce a poor decision boundary.
- Unscaled features can distort the SVM's margin and kernel calculations.

Debugging steps

- Verify labels, class encoding, data splits, missing values, and the full preprocessing pipeline.
- Compare against a simple baseline and inspect a confusion matrix to identify class-specific failures.
- Check training accuracy first: if it is also low, investigate model capacity, parameters, features, and data quality before focusing on generalization.

Optimization

- Scale features within a pipeline, fitting the scaler on training data only.
- Compare suitable kernels and tune C and gamma for an RBF kernel.
- Use cross-validated grid search or randomized search; consider class weights if classes are imbalanced.

Conclusion

Because both training and test performance are poor, first verify data and implementation, then tune model capacity and kernel settings.

12. K-Means — Highly Imbalanced Clusters

QUESTION

A K-Means clustering algorithm is applied to a dataset, but the clusters formed are highly imbalanced and do not reflect the actual data distribution. Investigate possible issues such as incorrect choice of K, poor initialization of centroids, or presence of outliers. Explain debugging approaches and suggest improvements like the Elbow method, K-Means++, and outlier removal techniques. **(5 Marks)**

ANSWER

Possible causes

- The chosen K may not match the natural grouping in the data.
- Poor centroid initialization can lead to a weak local solution.
- Outliers or features with large scales can pull centroids and distort cluster sizes. K-Means also tends to favor roughly spherical clusters and does not enforce balanced sizes.

Debugging steps

- Measure cluster sizes and inspect centroids, within-cluster distances, and outlying observations.
- Plot the data where possible and check feature scales and data quality.
- Repeat runs with different seeds to determine whether imbalance is stable or initialization-dependent.

Optimization

- Use the Elbow method to explore candidate values of K, supported by domain knowledge.
- Use K-Means++ and multiple initializations.
- Investigate and handle verified outliers, and scale features appropriately. If equal-sized clusters are a strict requirement, consider a method designed for balanced clustering.

Conclusion

The Elbow method and stronger initialization can help, but K-Means does not guarantee equal cluster sizes; the method must suit the data and objective.

13. RNN for Time-Series Prediction — Inconsistent Outputs

QUESTION

A recurrent neural network (RNN) designed for time-series prediction shows inconsistent outputs and fails to capture long-term dependencies. Analyze possible causes such as vanishing gradients, insufficient sequence length, or improper architecture design. Describe debugging steps and propose optimization techniques such as using LSTM/GRU units, gradient clipping, and sequence normalization. **(5 Marks)**

ANSWER

Possible causes

- Vanishing gradients can prevent learning from distant time steps.
- Input windows may be too short to contain the relevant historical pattern.

- Incorrect temporal ordering, unsuitable architecture, or inconsistent scaling can produce unreliable predictions.

Debugging steps

- Verify chronological ordering, missing timestamps, sampling intervals, and the construction of input/target windows.
- Plot predictions against actual values and inspect errors across forecast horizons.
- Monitor gradient norms and ensure that scaling is fitted on training data only; avoid random splits that leak future information.

Optimization

- Use LSTM or GRU units when longer dependencies are important.
- Apply gradient clipping to control exploding gradients.
- Normalize sequences consistently, choose a suitable window length, and tune architecture and learning rate using time-aware validation.

Conclusion

Correct temporal preprocessing, suitable sequence lengths, gated recurrent units, and stable gradients can improve forecasting consistency.

---

14. Naïve Bayes — Noisy Text Classification

QUESTION

A Naïve Bayes classifier used for text classification performs poorly when applied to real-world data containing noisy and irrelevant features. Examine possible issues such as violation of feature independence assumption, improper preprocessing, or imbalanced dataset. Explain debugging strategies and suggest improvements like feature selection, smoothing techniques, and data balancing methods. **(5 Marks)**

ANSWER

Possible causes

- Text may contain noise, inconsistent casing, irrelevant tokens, or poorly handled punctuation and missing values.
- Naïve Bayes assumes conditional independence of features given the class; correlated features can limit its effectiveness.
- Class imbalance or unseen words can lead to biased predictions or zero probabilities without suitable smoothing.

Debugging steps

- Inspect representative raw and processed text to verify tokenization, normalization, and label quality.
- Check class frequencies, confusion matrix, and per-class precision/recall rather than accuracy alone.
- Compare results with and without stop-word handling or other preprocessing, using a consistent validation split.

Optimization

- Remove irrelevant features and use an appropriate text representation, such as count or TF-IDF features depending on the chosen model.
- Apply Laplace/additive smoothing where appropriate to handle unseen features.
- Use class weighting or resampling when justified, and evaluate with stratified validation and class-sensitive metrics.

Conclusion

A clean text pipeline, relevant features, suitable smoothing, and attention to class imbalance can improve real-world text classification.

15. Reinforcement Learning — Erratic Navigation Agent

QUESTION

A reinforcement learning agent trained to solve a navigation problem fails to learn an optimal policy and shows erratic behavior. Analyze possible causes such as sparse rewards, improper exploration-exploitation balance, or incorrect reward function design. Describe debugging approaches and recommend optimization strategies like reward shaping, epsilon decay tuning, and policy optimization methods to improve learning performance. **(5 Marks)**

ANSWER

Possible causes

- Rewards may be too sparse for the agent to discover a route to the goal, or the reward function may encourage unintended behavior.
- Exploration may remain too high for too long or decay too quickly, preventing a good balance between exploration and exploitation.
- Incorrect state/action transitions or unsuitable learning parameters can also cause unstable behavior.

Debugging steps

- Log episode returns, success rate, route length, collisions or invalid moves, and action choices.
- Visualize sample trajectories and verify that state transitions, terminal conditions, and rewards match the navigation task.
- Compare learning curves across multiple seeds and check whether the agent visits the goal and relevant states.

Optimization

- Add carefully designed reward shaping for progress toward the goal while preserving the intended task objective.
- Tune epsilon decay so the agent explores early and increasingly exploits learned actions later.
- Tune algorithm-specific parameters and consider an appropriate policy-optimization method for the environment.

Conclusion

Correct environment logic, informative rewards, and a well-tuned exploration schedule are central to reliable navigation learning.